

SESSION FIREWALL

Complete Architecture, Behavioral Anomaly Detection, Unauthorized Access Tracking & Defensive Mastery Guide

System: Real-Time Zero-Trust Session Security & Geovelocity Firewall

Version: 1.0.0 Production Blueprint

Tech Stack: Node.js, Express, React, Tailwind CSS, MongoDB, JWT

Audience: Security Engineers, Full-Stack Architects & Financial DevOps

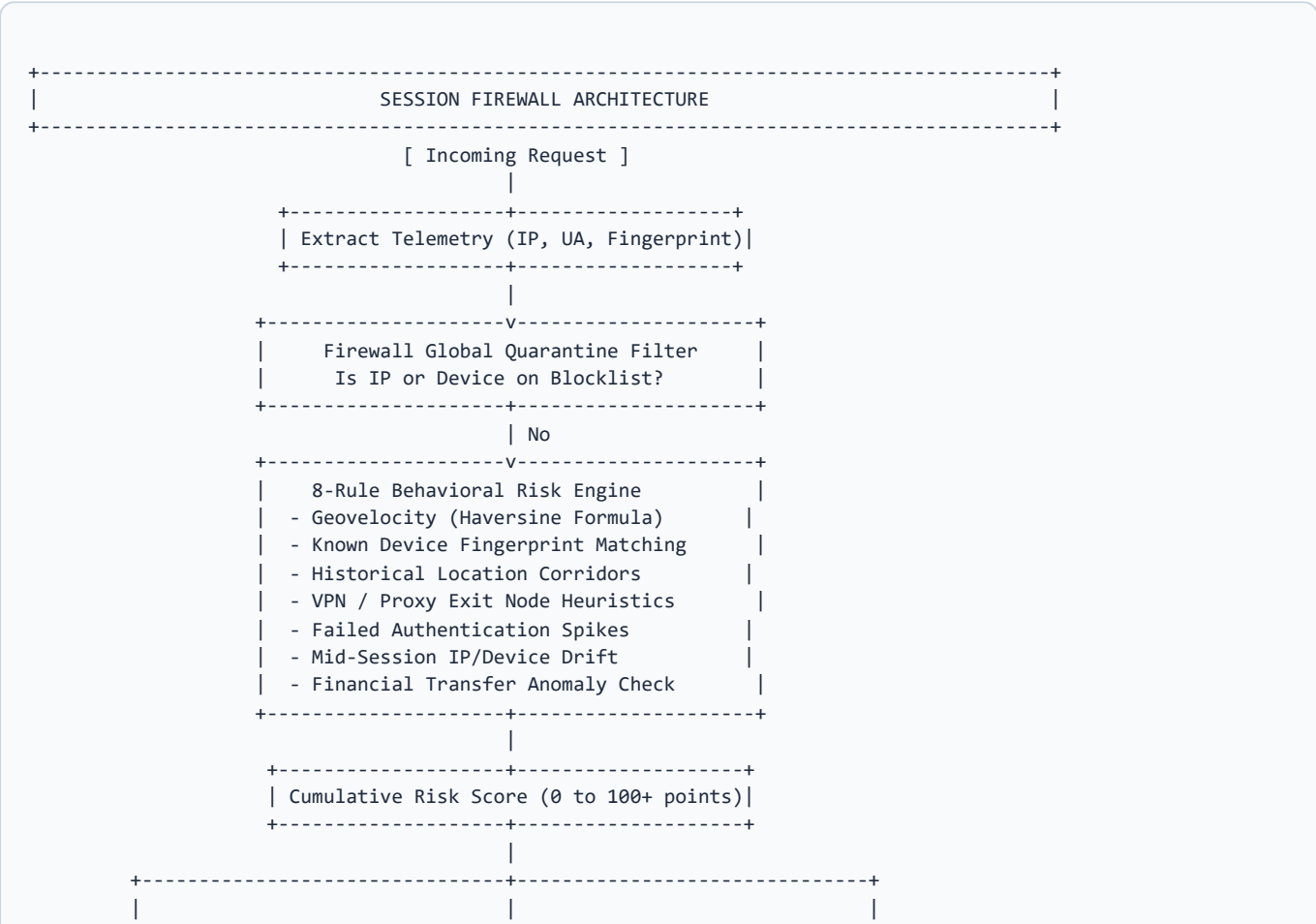
Published: September 2026

1. Executive Summary & Zero-Trust Paradigm

Modern digital banking architectures face an asymmetric threat landscape. Traditional authentication frameworks rely on static perimeter verification: once a customer enters their username and password, the resulting session token is typically treated as unconditionally trusted until expiration. This model fails against modern attack vectors, such as:

- **Session Token Hijacking:** Adversaries intercepting or stealing active tokens via compromised endpoints, local storage exfiltration, or proxy compromises.
- **Credential Stuffing & Botnet Attacks:** Automated attempts utilizing stolen credentials from third-party database breaches.
- **Geographic Proxy Spoofing:** Attackers routing traffic through foreign residential proxies or cloud hosting providers.
- **Concurrent Account Access:** Attackers authenticating simultaneously from distant countries while legitimate users are actively logged in.

The Session Firewall Principle: Authentication is never a one-time gate. In Session Firewall, every incoming request and token usage is evaluated continuously against an adaptive behavioral baseline. Telemetry anomalies instantly adjust the user's risk posture, automatically triggering step-up authentication, session termination, or global quarantine.



[0 - 29 Points]
LOW RISK
Session Allowed
Full Banking Access

[30 - 59 Points]
MEDIUM RISK
Step-Up MFA Required
Simulated OTP Verification

[60+ Points]
HIGH RISK
Session Quarantined
Transfers Restricted
Admin SOC Incident Logged
"Was This You?" Challenge

2. Session Telemetry Tracking Mechanics

To accurately differentiate legitimate customer actions from unauthorized intruders, Session Firewall captures multi-dimensional telemetry on every authentication and active request:

2.1 Client Telemetry Ingestion

The ingestion engine extracts client characteristics without requiring invasive client-side agents. The attributes are parsed from standard HTTP protocol headers:

- **User-Agent Parsing:** Utilizes `ua-parser-js` to decompose the raw header into structured components: browser family and version, operating system architecture, and device class (Desktop, Mobile, Tablet).
- **Device Signature Fingerprinting:** Combines Browser Name, OS Version, and architecture into a normalized fingerprint string (e.g., "Chrome 122 on Windows 11" or "Firefox 124 on macOS Sonoma").
- **IP Address Extraction:** Resolves client IP via `x-forwarded-for` headers (for reverse proxy architectures) with fallback to direct socket remote addresses.
- **Geographic Coordinates & Geocoding:** Resolves IP address to latitude, longitude, city, country, and ASN attributes.

2.2 The Geovelocity & Haversine Formula

To detect "impossible travel" (such as a session in Mumbai followed by a session in New York 10 minutes later), the system calculates great-circle distance using the **Haversine Formula**:

```
function calculateDistanceKm(lat1, lon1, lat2, lon2) {
  const R = 6371; // Earth's mean radius in kilometers
  const dLat = ((lat2 - lat1) * Math.PI) / 180;
  const dLon = ((lon2 - lon1) * Math.PI) / 180;

  const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos((lat1 * Math.PI) / 180) * Math.cos((lat2 * Math.PI) / 180) *
    Math.sin(dLon / 2) * Math.sin(dLon / 2);

  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c;
}

function calculateTravelSpeed(loc1, time1, loc2, time2) {
  const distanceKm = calculateDistanceKm(loc1.latitude, loc1.longitude, loc2.latitude, loc2.longitude);
  const elapsedHours = Math.abs(new Date(time2) - new Date(time1)) / (1000 * 60 * 60);
  return elapsedHours === 0 ? (distanceKm > 50 ? 999999 : 0) : (distanceKm / elapsedHours);
}
```

Commercial Airline Speed Boundary: Maximum commercial flight speed is approximately 800–900 km/h. If consecutive logins require an average ground velocity exceeding **800 km/h** across a distance greater than 200 km, the system flags an *Impossible Travel Anomaly* (+40 points).

3. Anomaly & Threat Detection Engine

The core risk engine evaluates sessions against 8 quantitative rules. Points accumulate into a composite threat index:

#	Threat Indicator	Points	Technical Detection Mechanism
1	New / Unrecognized Device	+30	Device fingerprint does not match any entry in <code>user.trustedDevices</code> or recent allowed sessions.
2	New IP Address	+15	IP address has not been observed in the user's historical allowed session baseline.
3	Unusual City or Country	+30	Current location is outside <code>user.trustedLocations</code> and not seen in prior legitimate logins.
4	VPN / Proxy / Datacenter IP	+15	IP address matches known Tor exit nodes, hosting provider ranges (AWS, DigitalOcean), or commercial VPN ranges.
5	Impossible Travel Velocity	+40	Haversine distance divided by elapsed time between consecutive sessions exceeds 800 km/h.
6	Failed Login Attempts (>= 3)	+25	Account experienced 3 or more consecutive bad password attempts prior to current authentication.
7	Mid-Session Telemetry Drift	+25	Active token was issued to Device A / IP A, but subsequent API calls originate from Device B or IP B.
8	Large / Unusual Transfer	+25	Outgoing transfer exceeds \$5,000 or exceeds 50% of the customer's total balance.

3.1 Risk Tiers & Automated Enforcement Actions

Score Tier	Classification	System Action	User Impact
0 – 29	LOW RISK	Allow Session	Normal access. Session token minted. Balance and transfers fully enabled.
30 – 59	MEDIUM RISK	Require MFA Verification	Session held in <code>MFA Required</code> state. Simulated 6-digit OTP dispatched to user. Access restricted until code verified.
60+	HIGH RISK	Quarantine & Lockdown	Session marked <code>Blocked</code> . Transfers locked on user account. In-app & simulated email notification sent. Incident logged in Admin SOC.

Attribution Disclaimer: In compliance with security best practices, the system identifies *suspicious and anomalous behavioral patterns* rather than claiming absolute identity attribution of an external attacker. Legitimate customers are given self-service verification ("Was this you?") to confirm or deny unexpected access.

4. How Unauthorized Access is Tested & Simulated

Security teams and QA engineers must verify that the Session Firewall reacts predictably to attack scenarios. The system includes built-in test runners and telemetry injection interfaces to test attacks safely:

4.1 Scenario 1: Safe Baseline Authentication

Simulates a legitimate user logging in from their home office workstation:

- **Device:** Chrome 122 on Windows 11 (Matches trusted device)
- **IP Address:** 103.21.244.0 (Mumbai, India)
- **Elapsed Time:** 5 hours since last session
- **Calculated Score:** 0 points → **LOW RISK**
- **System Behavior:** Access granted immediately. No MFA challenged. No security alerts.

4.2 Scenario 2: Impossible Travel & Credential Compromise Attack

Simulates an adversary using stolen credentials from a foreign server shortly after the legitimate user was active:

- **Baseline Session:** Alex Mercer authenticated in Mumbai, India 10 minutes ago.
- **Adversary Influx:** Login initiated from New York, United States (IP: 198.51.100.42) using Firefox 124 on macOS Sonoma .
- **Firewall Evaluation:**
 - Unrecognized Device Signature: **+30 pts**
 - New IP Address: **+15 pts**
 - Unusual Country / City: **+30 pts**
 - Impossible Travel (12,538 km in 10 mins → speed > 25,000 km/h): **+40 pts**
 - **Total Score: 115 points** → **HIGH RISK**
- **Automated Defensive Actions Executed:**
 1. Adversary session is immediately set to Blocked and rejected with HTTP 403 Forbidden .
 2. Customer account transfers are restricted: transfersLocked = true .
 3. In-app alert generated: "Suspicious login detected from Firefox on macOS in New York at 10:35 AM. Was this you?"
 4. Simulated outgoing email dispatched to customer's inbox.
 5. Security incident registered in Bank Admin SOC with status Blocked .

4.3 Scenario 3: Mid-Session Hijacking / Drift Simulation

Simulates token exfiltration where an adversary steals the JWT token and makes API calls from a different IP or device:

```
// Auth Middleware detects drift on incoming request:
const ipChanged = origSession.ipAddress !== currentRequestIp;
const deviceChanged = origSession.device !== currentRequestDevice;
```

```
if (ipChanged || deviceChanged) {  
  // Triggers Rule 7 (+25 points)  
  // If cumulative risk >= 60, session is immediately revoked mid-flight!  
}
```


5. Customer Security Lockdown Workflow

When an unauthorized login is flagged, the legitimate customer is presented with an urgent challenge modal and email notification:

In-App Alert Prompt:

"Suspicious login detected from Firefox 124 on macOS Sonoma in New York at 10:35 AM. Was this you?"

Options: [Yes, it was me] | [No, secure my account]

5.1 Workflow Execution on "No, secure my account"

When the customer confirms that the session was unauthorized, the backend initiates a comprehensive 5-step containment procedure:

```
[ Customer Clicks "No, secure my account" ]
      |
      v
1. REVOKE ATTACKER SESSION
   - Marked as "Confirmed Fraud"
   - Token invalidated immediately
      |
      v
2. TERMINATE ALL OTHER CONCURRENT SESSIONS
   - All non-trusted active sessions closed
   - Preserves only the user's current verified session
      |
      v
3. POPULATE GLOBAL QUARANTINE BLOCKLIST
   - Adversary IP (198.51.100.42) added to Blocklist
   - Adversary Device fingerprint quarantined
      |
      v
4. LOCK OUTGOING WIRE & ACH TRANSFERS
   - user.transfersLocked = true
   - Attempts to transfer return 403 Forbidden
      |
      v
5. NOTIFY BANK SOC & DISPATCH INSTRUCTIONS
   - Admin SOC alert updated to "Confirmed Fraud"
   - Simulated email sent with password reset guidance
```

5.2 Workflow Execution on "Yes, it was me"

If the customer was legitimately using a foreign VPN or new laptop:

- The session status is transitioned to `Resolved`.
- The device signature is added to `user.trustedDevices` so future logins will not trigger unnecessary friction.
- Transfer locks resulting from the alert are cleared.
- Admin SOC logs the event as customer-verified.

6. Bank Security Operations Center (SOC)

The Bank Security Administrator console provides comprehensive visibility and manual override authority over all security telemetry:

6.1 Key SOC Metrics & Telemetry Feeds

- **Total Evaluated Sessions:** Aggregated throughput of all authentication events across the bank.
- **Suspicious / MFA Flagged:** Sessions requiring step-up authentication (Medium Risk: 30–59 pts).
- **Blocked & Confirmed Fraud:** High-risk attacks halted by the firewall or reported by customers.
- **Quarantined Artifacts:** Real-time count of blocked IPs and device signatures.

6.2 7-Day Threat Visualizer

An interactive SVG visualization plotting daily volumes:

- **Blue Bars:** Total login volume across the institution.
- **Amber Bars:** Telemetry deviations flagged as Medium Risk.
- **Red Bars:** Hard-blocked attacks and confirmed fraud reports.

6.3 Administrator Remediation Controls

Administrative Action	Endpoint	Operational Effect
Block / Unblock IP	POST /api/admin/blocklist/toggle	Instantly quarantines or restores an IP address globally across all accounts.
Block / Unblock Device	POST /api/admin/blocklist/toggle	Adds or removes a hardware/browser fingerprint from the firewall blacklist.
Unlock Customer Transfers	PATCH /api/admin/users/:id/transfers	Restores wire and ACH transfer capabilities after customer identity verification.
Update Session Status	PATCH /api/admin/sessions/:id/status	Overrides session state to Resolved , Blocked , or Confirmed Fraud .

7. REST API & Data Schema Reference

7.1 Data Models Schema Summary

User Model (`models/User.js`)

```
{
  name: String,
  email: { type: String, unique: true },
  password: String, // Bcrypt hash
  role: { type: String, enum: ['customer', 'admin'] },
  accountNumber: String,
  balance: Number, // Default $24,850.00
  transfersLocked: Boolean,
  transferLockReason: String,
  failedLoginAttempts: Number,
  trustedDevices: [String],
  trustedLocations: [{ city: String, country: String }]
}
```

Session Model (`models/Session.js`)

```
{
  userId: ObjectId,
  sessionId: { type: String, unique: true },
  device: String,
  browser: String,
  os: String,
  ipAddress: String,
  location: { city: String, country: String, latitude: Number, longitude: Number },
  isVpn: Boolean,
  riskScore: Number,
  riskReasons: [String],
  riskLevel: { type: String, enum: ['low', 'medium', 'high'] },
  status: { type: String, enum: ['Allowed', 'MFA Required', 'Blocked', 'Confirmed Fraud', 'Resolved'] },
  isActive: Boolean,
  revokedAt: Date,
  revokedReason: String
}
```

7.2 Core Endpoints

Method	Endpoint	Auth Required	Description
POST	<code>/api/auth/login</code>	None	Evaluates credentials, extracts telemetry, calculates risk, and issues JWT or MFA challenge.

Method	Endpoint	Auth Required	Description
POST	/api/auth/verify-mfa	None	Validates simulated 6-digit OTP and activates medium-risk session.
GET	/api/sessions	JWT (Customer)	Returns all active/historical sessions with telemetry and current session flag.
POST	/api/sessions/mark-not-me	JWT (Customer)	Executes customer denial: quarantines IP/device, locks transfers, alerts admin.
POST	/api/bank/transfer	JWT (Customer)	Executes funds transfer with transfer lock check and financial risk policy.
GET	/api/admin/sessions	JWT (Admin)	Filterable security incident audit trail.
POST	/api/demo/scenario-suspicious	None	Executes 1-click impossible travel attack scenario for testing.

8. Enterprise Hardening & Operational Runbook

For production deployment in Tier-1 financial institutions, the Session Firewall architecture can be further enhanced with the following enterprise controls:

8.1 Redis Distributed Cache & In-Memory Rate Limiting

In high-throughput environments processing tens of thousands of logins per second:

- Store the active **Quarantine Blocklist** in a Redis in-memory cache or Redis Bloom Filter for sub-millisecond lookup latency.
- Implement token bucket rate limiting per IP and per account to mitigate credential stuffing before invoking database queries.

8.2 Commercial GeoIP & Threat Intelligence Feeds

Integrate enterprise-grade intelligence providers:

- **MaxMind GeoIP2 Precision / IPinfo:** For verified geographic coordinates and cellular connection detection.
- **Threat Intelligence APIs (Spur.us, IPQualityScore):** For real-time detection of residential proxy networks and commercial VPNs.

8.3 Hardware Security Keys & FIDO2 / WebAuthn

For accounts requiring elevated security (corporate banking, wire desks):

- Supplement simulated OTP with WebAuthn (FIDO2 / YubiKey) challenges on medium-risk sessions to completely eliminate phishing vulnerabilities.

Mastery Summary: Session Firewall transforms traditional static banking authentication into a dynamic, zero-trust defense network. By continuously scoring device signatures, physical velocity, and financial patterns, the platform stops unauthorized access in real-time while preserving a frictionless experience for legitimate customers.